

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2
За седьмой семестр
По дисциплине: «Интеллектуальный анализ данных »
Тема: «Автоэнкодеры»

Выполнил:
Студент 4 курса
Группы ИИ-26
Хомиченко И.А.
Проверил:
Никитин С.К.

Цель: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 7

№	Выборка	Класс
7	Mushroom	poisonous

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, trustworthiness
import sklearn
import warnings

warnings.filterwarnings('ignore')

np.random.seed(42)
torch.manual_seed(42)

print("scikit-learn version:", sklearn.__version__)

print("=" * 60)
print("1. DATA LOADING")
print("=" * 60)

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data"

columns = [
    "class",
```

```

        "cap-shape",
        "cap-surface",
        "cap-color",
        "bruises",
        "odor",
        "gill-attachment",
        "gill-spacing",
        "gill-size",
        "gill-color",
        "stalk-shape",
        "stalk-root",
        "stalk-surface-above-ring",
        "stalk-surface-below-ring",
        "stalk-color-above-ring",
        "stalk-color-below-ring",
        "veil-type",
        "veil-color",
        "ring-number",
        "ring-type",
        "spore-print-color",
        "population",
        "habitat"
    ]

mushrooms = pd.read_csv(url, header=None, names=columns)

print(f"Dataset shape: {mushrooms.shape}")
print(f"Columns: {list(mushrooms.columns)}")
print("\nClass distribution:")
print(mushrooms["class"].value_counts())

X = mushrooms.drop(columns=["class"])
y_raw = mushrooms["class"].values

y = np.array([1 if value == "p" else 0 for value in y_raw])

encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_encoded = encoder.fit_transform(X)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_encoded)

print(f"\nEncoded dataset shape: {X_scaled.shape}")

unique, counts = np.unique(y, return_counts=True)

print("\nClass distribution:")
for u, c in zip(unique, counts):
    print(f"  Class {u}: {c} samples")

class_names = {
    0: "Edible",
    1: "Poisonous"
}

colors = ["#2ecc71", "#e74c3c"]
markers = ["o", "^"]

print("\n" + "=" * 60)
print("2. AUTOENCODER ARCHITECTURE SEARCH")
print("=" * 60)

X_tensor = torch.FloatTensor(X_scaled)

```

```

dataset = TensorDataset(X_tensor, X_tensor)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

input_dim = X_scaled.shape[1]

class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim, hidden_dim=64):
        super(Autoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

    def encode(self, x):
        return self.encoder(x)

def train_autoencoder(model, dataloader, epochs=100, lr=1e-3):
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    losses = []
    model.train()

    for epoch in range(epochs):
        epoch_loss = 0.0

        for batch_x, batch_y in dataloader:
            optimizer.zero_grad()

            output = model(batch_x)

            loss = criterion(output, batch_y)

            loss.backward()
            optimizer.step()

            epoch_loss += loss.item() * batch_x.size(0)

        epoch_loss /= len(dataloader.dataset)
        losses.append(epoch_loss)

        if (epoch + 1) % 20 == 0:
            print(
                f" Epoch [{epoch + 1}/{epochs}], "
                f"Loss: {epoch_loss:.6f}"
            )

    return losses

```

```

def get_latent_representation(model, X_tensor):
    model.eval()

    with torch.no_grad():
        latent = model.encode(X_tensor).numpy()

    return latent

configs = [
    {
        "hidden_dim": 32,
        "lr": 1e-3,
        "epochs": 100
    },
    {
        "hidden_dim": 64,
        "lr": 1e-3,
        "epochs": 100
    },
    {
        "hidden_dim": 128,
        "lr": 1e-3,
        "epochs": 100
    }
]

print("\n--- Architecture search for latent_dim = 2 ---")

search_results_2d = {}

for cfg in configs:
    print(
        f"\n Config: "
        f"hidden_dim={cfg['hidden_dim']}, "
        f"lr={cfg['lr']}, "
        f"epochs={cfg['epochs']}"
    )

    ae = Autoencoder(
        input_dim=input_dim,
        latent_dim=2,
        hidden_dim=cfg["hidden_dim"]
    )

    losses = train_autoencoder(
        ae,
        dataloader,
        epochs=cfg["epochs"],
        lr=cfg["lr"]
    )

    search_results_2d[cfg["hidden_dim"]] = losses[-1]

    print(f" Final loss: {losses[-1]:.6f}")

print("\n--- Architecture search for latent_dim = 3 ---")

search_results_3d = {}

for cfg in configs:
    print(
        f"\n Config: "

```

```

        f"hidden_dim={cfg['hidden_dim']}, "
        f"lr={cfg['lr']}, "
        f"epochs={cfg['epochs']}"
    )

    ae = Autoencoder(
        input_dim=input_dim,
        latent_dim=3,
        hidden_dim=cfg["hidden_dim"]
    )

    losses = train_autoencoder(
        ae,
        dataloader,
        epochs=cfg["epochs"],
        lr=cfg["lr"]
    )

    search_results_3d[cfg["hidden_dim"]] = losses[-1]

    print(f"    Final loss: {losses[-1]:.6f}")

best_hidden_2d = min(
    search_results_2d,
    key=search_results_2d.get
)

best_hidden_3d = min(
    search_results_3d,
    key=search_results_3d.get
)

print(
    f"\nBest hidden_dim for latent_dim=2: "
    f"{best_hidden_2d} "
    f"(loss={search_results_2d[best_hidden_2d]:.6f})"
)

print(
    f"Best hidden_dim for latent_dim=3: "
    f"{best_hidden_3d} "
    f"(loss={search_results_3d[best_hidden_3d]:.6f})"
)

print("\n--- Training final autoencoder with 2 latent neurons ---")

ae_2d = Autoencoder(
    input_dim=input_dim,
    latent_dim=2,
    hidden_dim=best_hidden_2d
)

losses_2d = train_autoencoder(
    ae_2d,
    dataloader,
    epochs=100,
    lr=1e-3
)

latent_2d = get_latent_representation(
    ae_2d,
    X_tensor
)

```

```

print("\n--- Training final autoencoder with 3 latent neurons ---")

ae_3d = Autoencoder(
    input_dim=input_dim,
    latent_dim=3,
    hidden_dim=best_hidden_3d
)

losses_3d = train_autoencoder(
    ae_3d,
    dataloader,
    epochs=100,
    lr=1e-3
)

latent_3d = get_latent_representation(
    ae_3d,
    X_tensor
)

print("\nAutoencoders trained.")

print("\n" + "=" * 60)
print("3. PCA")
print("=" * 60)

pca_2d = PCA(
    n_components=2,
    random_state=42
)

pca_2d_result = pca_2d.fit_transform(X_scaled)

print(
    f"PCA 2D: explained variance = "
    f"{pca_2d.explained_variance_ratio_.sum():.4f}"
)

pca_3d = PCA(
    n_components=3,
    random_state=42
)

pca_3d_result = pca_3d.fit_transform(X_scaled)

print(
    f"PCA 3D: explained variance = "
    f"{pca_3d.explained_variance_ratio_.sum():.4f}"
)

print("\n" + "=" * 60)
print("4. t-SNE")
print("=" * 60)

perplexity_values = [
    20,
    30,
    40,
    50,
    60
]

tsne_2d_results = {}
tsne_3d_results = {}

```

```

np.random.seed(42)

sample_indices = np.random.choice(
    len(X_scaled),
    size=min(1500, len(X_scaled)),
    replace=False
)

X_sample = X_scaled[sample_indices]
y_sample = y[sample_indices]

print(
    f"Subsample used: "
    f"{len(sample_indices)} samples"
)

for perp in perplexity_values:
    print(f"\n Perplexity = {perp}")

    tsne_2d = TSNE(
        n_components=2,
        perplexity=perp,
        init="pca",
        random_state=42,
        learning_rate="auto",
        max_iter=1000
    )

    tsne_2d_result = tsne_2d.fit_transform(
        X_sample
    )

    tsne_2d_results[perp] = tsne_2d_result

    print("    2D done")

    tsne_3d = TSNE(
        n_components=3,
        perplexity=perp,
        init="pca",
        random_state=42,
        learning_rate="auto",
        max_iter=1000
    )

    tsne_3d_result = tsne_3d.fit_transform(
        X_sample
    )

    tsne_3d_results[perp] = tsne_3d_result

    print("    3D done")

print("\nt-SNE completed.")

print("\n" + "=" * 60)
print("5. SELECTING BEST PERPLEXITY BY TRUSTWORTHINESS")
print("=" * 60)

trust_scores = {}

for perp in perplexity_values:
    score = trustworthiness(

```

```

        X_sample,
        tsne_2d_results[perp],
        n_neighbors=5
    )

    trust_scores[perp] = score

    print(
        f"    Perplexity = {perp}: "
        f"trustworthiness = {score:.4f}"
    )

best_perp = max(
    trust_scores,
    key=trust_scores.get
)

print(
    f"\nBest perplexity: {best_perp} "
    f"(trustworthiness = "
    f"{trust_scores[best_perp]:.4f})"
)

print("\n" + "=" * 60)
print("6. VISUALIZATION")
print("=" * 60)

fig, ax = plt.subplots(figsize=(10, 8))

for cls in np.unique(y):
    mask = y == cls

    ax.scatter(
        latent_2d[mask, 0],
        latent_2d[mask, 1],
        c=colors[cls],
        marker=markers[cls],
        label=class_names[cls],
        alpha=0.6,
        s=20,
        edgecolors="w",
        linewidth=0.3
    )

ax.set_title(
    "Autoencoder (2D latent) - Mushroom",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel("Latent Dimension 1")
ax.set_ylabel("Latent Dimension 2")

ax.legend(
    title="Mushroom Class",
    loc="best"
)

ax.grid(
    True,
    alpha=0.3
)

plt.tight_layout()

```

```

plt.savefig(
    "mushroom_ae_2d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

fig = plt.figure(
    figsize=(12, 10)
)

ax = fig.add_subplot(
    111,
    projection="3d"
)

for cls in np.unique(y):
    mask = y == cls

    ax.scatter(
        latent_3d[mask, 0],
        latent_3d[mask, 1],
        latent_3d[mask, 2],
        c=colors[cls],
        marker=markers[cls],
        label=class_names[cls],
        alpha=0.6,
        s=20,
        edgecolors="w",
        linewidth=0.3
    )

ax.set_title(
    "Autoencoder (3D latent) - Mushroom",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel("Latent 1")
ax.set_ylabel("Latent 2")
ax.set_zlabel("Latent 3")

ax.legend(
    title="Mushroom Class",
    loc="best"
)

plt.tight_layout()

plt.savefig(
    "mushroom_ae_3d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

fig, ax = plt.subplots(
    figsize=(10, 8)
)

for cls in np.unique(y):

```

```

mask = y == cls

ax.scatter(
    pca_2d_result[mask, 0],
    pca_2d_result[mask, 1],
    c=colors[cls],
    marker=markers[cls],
    label=class_names[cls],
    alpha=0.6,
    s=20,
    edgecolors="w",
    linewidth=0.3
)

ax.set_title(
    "PCA (2 components) - Mushroom",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel(
    f"PC1 "
    f"({pca_2d.explained_variance_ratio_[0]:.2%})"
)

ax.set_ylabel(
    f"PC2 "
    f"({pca_2d.explained_variance_ratio_[1]:.2%})"
)

ax.legend(
    title="Mushroom Class",
    loc="best"
)

ax.grid(
    True,
    alpha=0.3
)

plt.tight_layout()

plt.savefig(
    "mushroom_pca_2d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

fig = plt.figure(
    figsize=(12, 10)
)

ax = fig.add_subplot(
    111,
    projection="3d"
)

for cls in np.unique(y):
    mask = y == cls

    ax.scatter(
        pca_3d_result[mask, 0],

```

```

        pca_3d_result[mask, 1],
        pca_3d_result[mask, 2],
        c=colors[cls],
        marker=markers[cls],
        label=class_names[cls],
        alpha=0.6,
        s=20,
        edgecolors="w",
        linewidth=0.3
    )

ax.set_title(
    "PCA (3 components) - Mushroom",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel(
    f"PC1 "
    f"({pca_3d.explained_variance_ratio_[0]:.2%})"
)

ax.set_ylabel(
    f"PC2 "
    f"({pca_3d.explained_variance_ratio_[1]:.2%})"
)

ax.set_zlabel(
    f"PC3 "
    f"({pca_3d.explained_variance_ratio_[2]:.2%})"
)

ax.legend(
    title="Mushroom Class",
    loc="best"
)

plt.tight_layout()

plt.savefig(
    "mushroom_pca_3d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

fig, axes = plt.subplots(
    1,
    len(perplexity_values),
    figsize=(4 * len(perplexity_values), 4)
)

for idx, perp in enumerate(
    perplexity_values
):
    ax = axes[idx]

    result = tsne_2d_results[perp]

    for cls in np.unique(y_sample):
        mask = y_sample == cls

        ax.scatter(

```

```

        result[mask, 0],
        result[mask, 1],
        c=colors[cls],
        marker=markers[cls],
        alpha=0.6,
        s=15,
        edgecolors="w",
        linewidth=0.2
    )

    ax.set_title(
        f"Perplexity = {perp}",
        fontsize=11
    )

    ax.set_xticks([])
    ax.set_yticks([])

fig.suptitle(
    "t-SNE (2D) - Mushroom, various perplexity",
    fontsize=14,
    fontweight="bold"
)

plt.tight_layout()

plt.savefig(
    "mushroom_tsne_2d_perplexity.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

result_3d = tsne_3d_results[
    best_perp
]

fig = plt.figure(
    figsize=(12, 10)
)

ax = fig.add_subplot(
    111,
    projection="3d"
)

for cls in np.unique(y_sample):
    mask = y_sample == cls

    ax.scatter(
        result_3d[mask, 0],
        result_3d[mask, 1],
        result_3d[mask, 2],
        c=colors[cls],
        marker=markers[cls],
        label=class_names[cls],
        alpha=0.6,
        s=20,
        edgecolors="w",
        linewidth=0.3
    )

ax.set_title(

```

```

        f"t-SNE (3D, perplexity={best_perp}) - Mushroom",
        fontsize=14,
        fontweight="bold"
    )

ax.set_xlabel("t-SNE 1")
ax.set_ylabel("t-SNE 2")
ax.set_zlabel("t-SNE 3")

ax.legend(
    title="Mushroom Class",
    loc="best"
)

plt.tight_layout()

plt.savefig(
    "mushroom_tsne_3d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

fig, axes = plt.subplots(
    1,
    3,
    figsize=(18, 5)
)

ax = axes[0]

for cls in np.unique(y):
    mask = y == cls

    ax.scatter(
        latent_2d[mask, 0],
        latent_2d[mask, 1],
        c=colors[cls],
        marker=markers[cls],
        alpha=0.5,
        s=15
    )

ax.set_title(
    "Autoencoder (2D)",
    fontsize=12,
    fontweight="bold"
)

ax.set_xlabel("Latent 1")
ax.set_ylabel("Latent 2")

ax.grid(
    True,
    alpha=0.3
)

ax = axes[1]

for cls in np.unique(y):
    mask = y == cls

    ax.scatter(

```

```

        pca_2d_result[mask, 0],
        pca_2d_result[mask, 1],
        c=colors[cls],
        marker=markers[cls],
        alpha=0.5,
        s=15
    )

ax.set_title(
    "PCA (2D)",
    fontsize=12,
    fontweight="bold"
)

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")

ax.grid(
    True,
    alpha=0.3
)

ax = axes[2]

result = tsne_2d_results[
    best_perp
]

for cls in np.unique(y_sample):
    mask = y_sample == cls

    ax.scatter(
        result[mask, 0],
        result[mask, 1],
        c=colors[cls],
        marker=markers[cls],
        alpha=0.5,
        s=15
    )

ax.set_title(
    f"t-SNE (2D, perplexity={best_perp})",
    fontsize=12,
    fontweight="bold"
)

ax.set_xlabel("t-SNE 1")
ax.set_ylabel("t-SNE 2")

ax.grid(
    True,
    alpha=0.3
)

handles = [
    plt.Line2D(
        [0],
        [0],
        marker=markers[i],
        color="w",
        markerfacecolor=colors[i],
        markersize=8,
        label=class_names[i]
    )

```

```

        for i in range(2)
    ]

fig.legend(
    handles=handles,
    title="Mushroom Class",
    loc="lower center",
    ncol=2,
    bbox_to_anchor=(0.5, -0.05)
)

fig.suptitle(
    "Comparison of dimensionality reduction methods - Mushroom",
    fontsize=14,
    fontweight="bold",
    y=1.02
)

plt.tight_layout()

plt.savefig(
    "mushroom_comparison_2d.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print(
    "\nAll visualizations saved "
    "to the current directory."
)

print("Done!")

```

Результат программы:

1. DATA LOADING

Dataset shape: (8124, 23)

Columns: ['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor', 'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color', 'stalk-shape', 'stalk-root', 'stalk-surface-above-ring', 'stalk-surface-below-ring', 'stalk-color-above-ring', 'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number', 'ring-type', 'spore-print-color', 'population', 'habitat']

Class distribution:

class

e 4208

p 3916

Name: count, dtype: int64

Encoded dataset shape: (8124, 117)

Class distribution:

Class 0: 4208 samples

Class 1: 3916 samples

2. AUTOENCODER ARCHITECTURE SEARCH

--- Architecture search for latent_dim = 2 ---

Config: hidden_dim=32, lr=0.001, epochs=100

Epoch [20/100], Loss: 0.455847

Epoch [40/100], Loss: 0.418459

Epoch [60/100], Loss: 0.401952

Epoch [80/100], Loss: 0.394540

Epoch [100/100], Loss: 0.389016

Final loss: 0.389016

Config: hidden_dim=64, lr=0.001, epochs=100

Epoch [20/100], Loss: 0.428884

Epoch [40/100], Loss: 0.382240

Epoch [60/100], Loss: 0.362901

Epoch [80/100], Loss: 0.348771

Epoch [100/100], Loss: 0.338039

Final loss: 0.338039

Config: hidden_dim=128, lr=0.001, epochs=100

Epoch [20/100], Loss: 0.405556

Epoch [40/100], Loss: 0.361206

Epoch [60/100], Loss: 0.336296
Epoch [80/100], Loss: 0.323531
Epoch [100/100], Loss: 0.314497
Final loss: 0.314497

--- Architecture search for latent_dim = 3 ---

Config: hidden_dim=32, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.415719
Epoch [40/100], Loss: 0.385600
Epoch [60/100], Loss: 0.367358
Epoch [80/100], Loss: 0.356199
Epoch [100/100], Loss: 0.348187
Final loss: 0.348187

Config: hidden_dim=64, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.350250
Epoch [40/100], Loss: 0.305291
Epoch [60/100], Loss: 0.280865
Epoch [80/100], Loss: 0.270802
Epoch [100/100], Loss: 0.263207
Final loss: 0.263207

Config: hidden_dim=128, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.316471
Epoch [40/100], Loss: 0.268863
Epoch [60/100], Loss: 0.249534
Epoch [80/100], Loss: 0.235798
Epoch [100/100], Loss: 0.223044

Final loss: 0.223044

Best hidden_dim for latent_dim=2: 128 (loss=0.314497)

Best hidden_dim for latent_dim=3: 128 (loss=0.223044)

--- Training final autoencoder with 2 latent neurons ---

Epoch [20/100], Loss: 0.425294

Epoch [40/100], Loss: 0.376376

Epoch [60/100], Loss: 0.353270

Epoch [80/100], Loss: 0.337515

Epoch [100/100], Loss: 0.326865

--- Training final autoencoder with 3 latent neurons ---

Epoch [20/100], Loss: 0.313749

Epoch [40/100], Loss: 0.271539

Epoch [60/100], Loss: 0.251006

Epoch [80/100], Loss: 0.236583

Epoch [100/100], Loss: 0.223694

Autoencoders trained.

3. PCA

PCA 2D: explained variance = 0.1702

PCA 3D: explained variance = 0.2422

4. t-SNE

Subsample used: 1500 samples

Perplexity = 20

2D done

3D done

Perplexity = 30

2D done

3D done

Perplexity = 40

2D done

3D done

Perplexity = 50

2D done

3D done

Perplexity = 60

2D done

3D done

t-SNE completed.

5. SELECTING BEST PERPLEXITY BY TRUSTWORTHINESS

Perplexity = 20: trustworthiness = 0.9959

Perplexity = 30: trustworthiness = 0.9956

Perplexity = 40: trustworthiness = 0.9948

Perplexity = 50: trustworthiness = 0.9944

Perplexity = 60: trustworthiness = 0.9951

Best perplexity: 20 (trustworthiness = 0.9959)

Figure 1

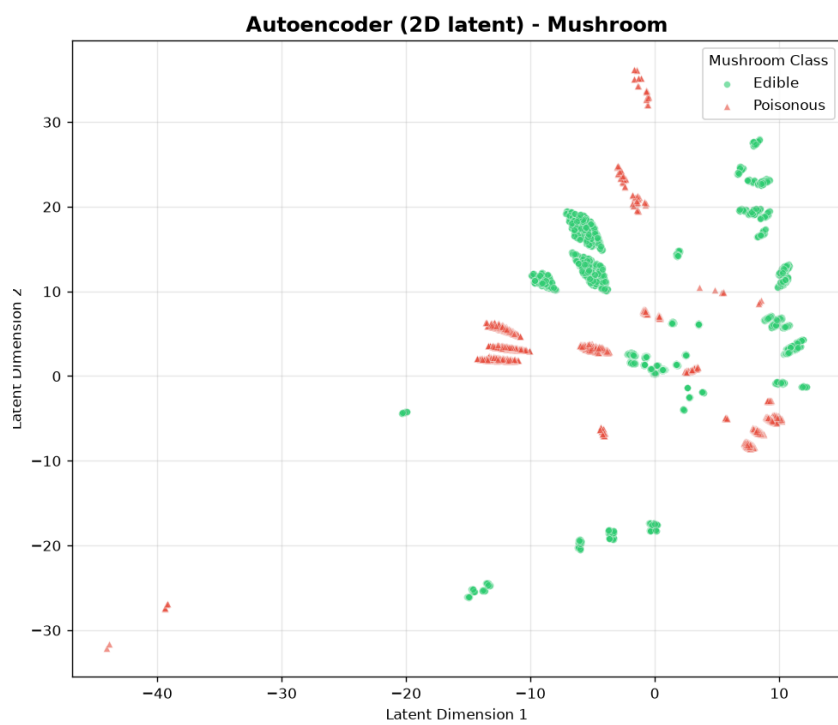
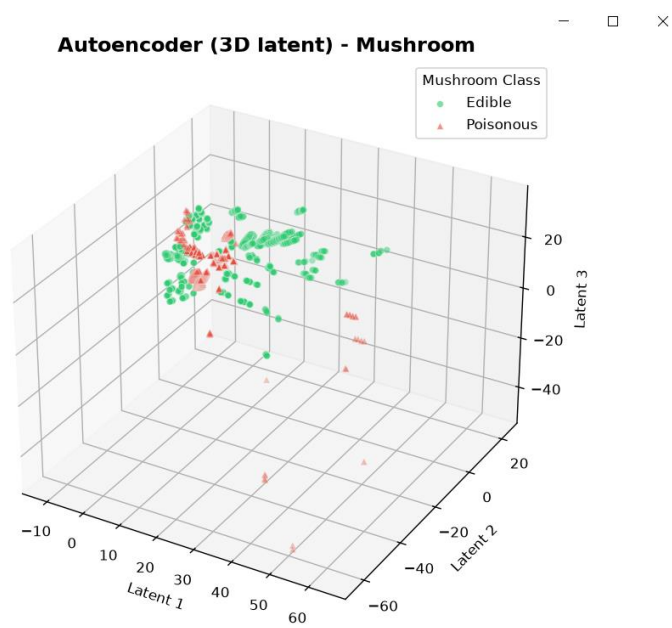


Figure 1



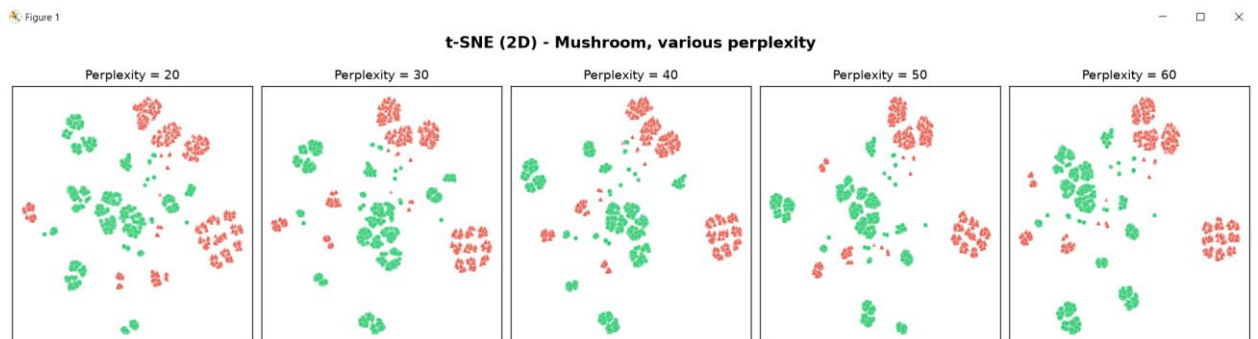
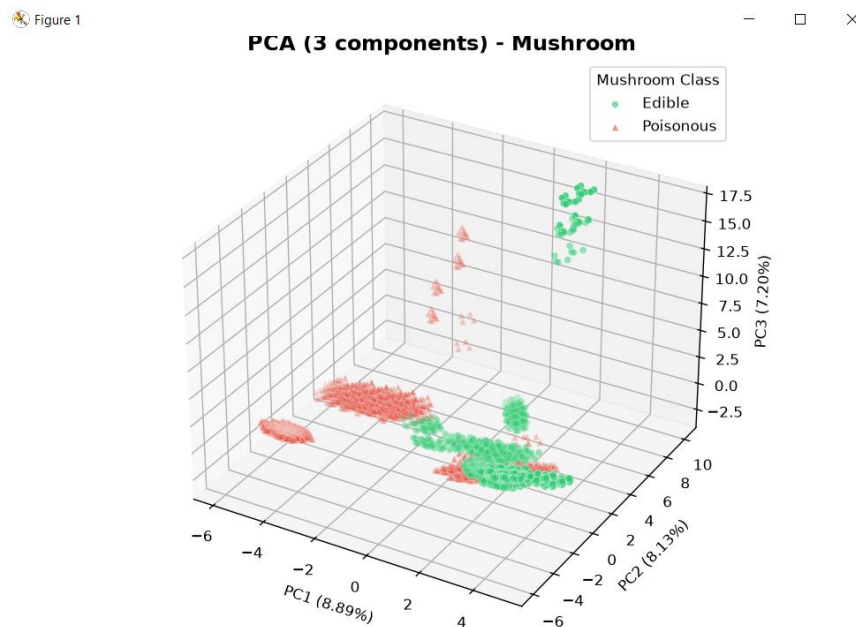
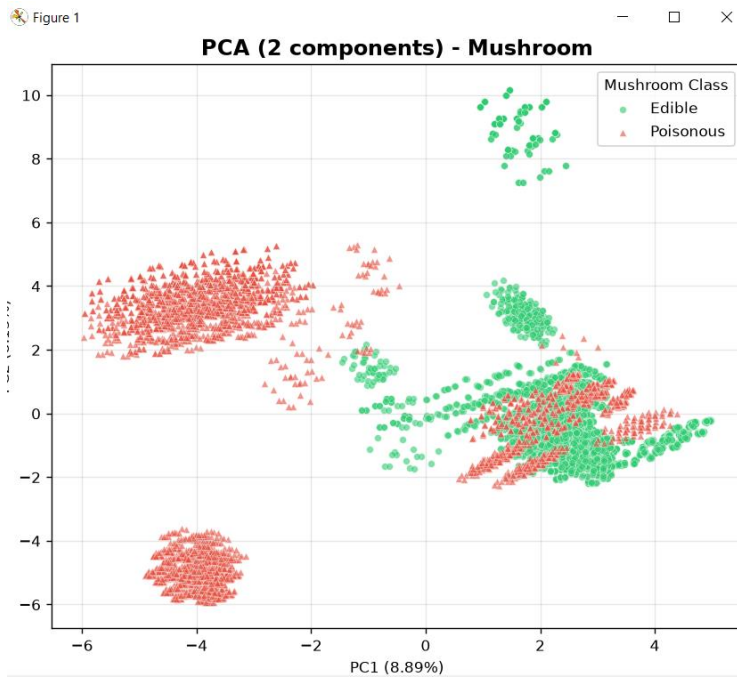
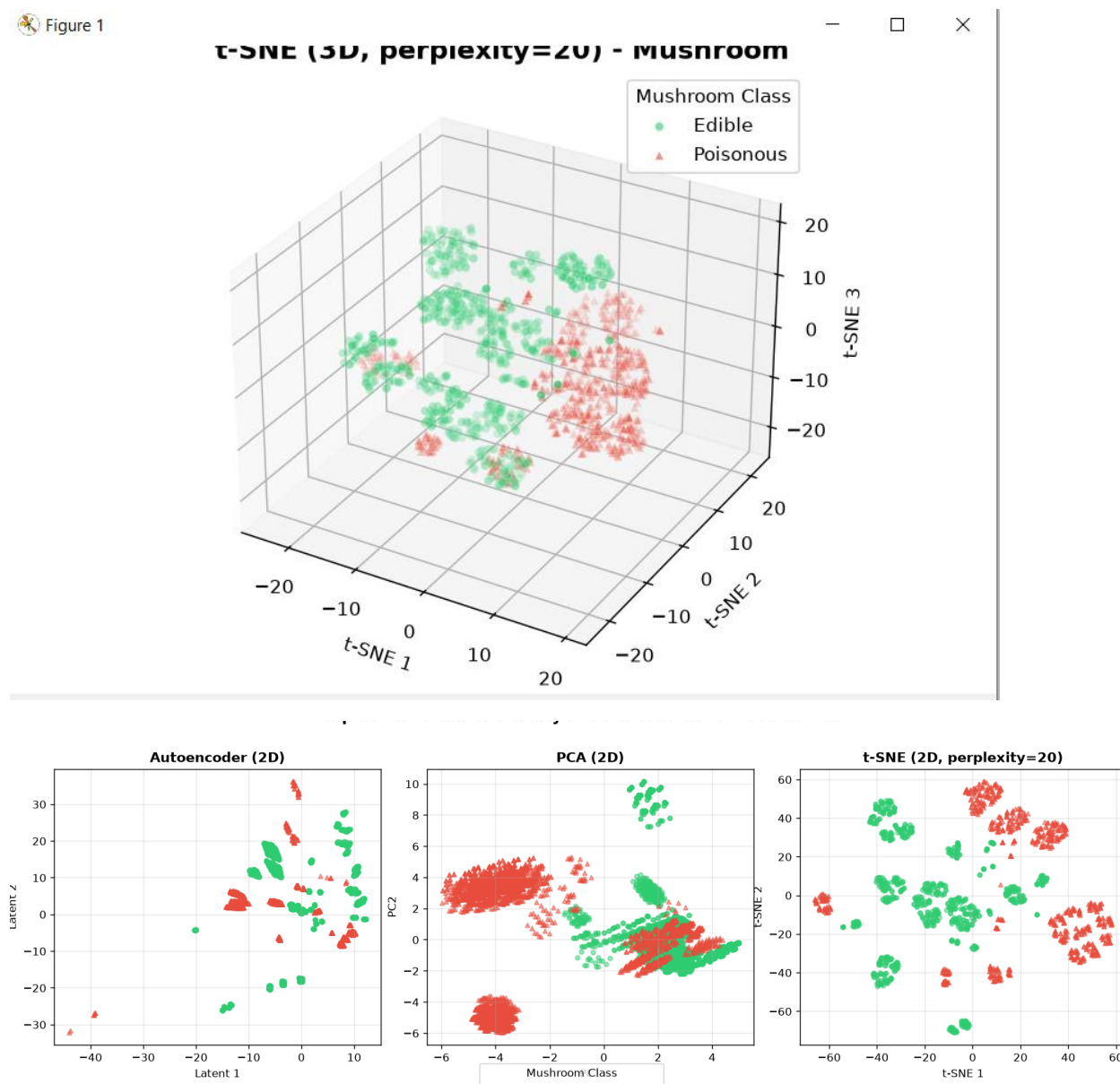


Figure 1



В работе выполнена визуализация данных Mushroom: 8124 образца, 22 исходных категориальных признака и целевая переменная class, разделяющая грибы на два класса — съедобные (Edible) и ядовитые (Poisonous). В выборке 4208 съедобных и 3916 ядовитых грибов, поэтому выраженного дисбаланса классов практически нет. После кодирования категориальных признаков методом One-Hot Encoding размерность пространства признаков увеличилась до 117.

Автоэнкодер обучался экспериментально с подбором размера скрытого слоя 32, 64 и 128 нейронов. Лучший результат для двумерного и трёхмерного латентного пространства показала конфигурация со скрытым слоем из 128 нейронов. При поиске архитектуры ошибка восстановления составила около 0.3145 для двух латентных нейронов и 0.2230 для трёх. При финальном обучении были получены значения около 0.3269 для 2D и 0.2237 для 3D. Это показывает, что трёхмерное латентное представление лучше сохраняет информацию исходного набора данных. На графиках автоэнкодера заметно

частичное разделение съедобных и ядовитых грибов, однако некоторые группы классов пересекаются.

Метод PCA показал значительно более слабое сохранение исходной информации: первые две главные компоненты объясняют только 17.02% дисперсии, а три компоненты — 24.22%. На 2D- и 3D-графиках видны отдельные группы съедобных и ядовитых грибов, однако значительная часть объектов пересекается. Это говорит о том, что линейное снижение размерности PCA недостаточно хорошо описывает сложную структуру данного набора категориальных данных.

Наиболее выраженное разделение классов показал метод t-SNE. Были проверены значения perplexity 20, 30, 40, 50 и 60. Лучший результат по метрике trustworthiness получен при perplexity = 20, где значение составило 0.9959. Для остальных значений perplexity показатель также оставался очень высоким — примерно от 0.9944 до 0.9956. На двумерной и трёхмерной визуализации t-SNE формируются хорошо различимые локальные кластеры, многие из которых преимущественно относятся либо к съедобным, либо к ядовитым грибам.

Итог: для набора данных Mushroom нелинейные методы снижения размерности показали себя лучше PCA. t-SNE обеспечивает наиболее наглядное разделение классов и лучше подходит для исследовательской визуализации структуры данных. Автоэнкодер также хорошо выделяет скрытые зависимости и при трёх латентных нейронах показывает более низкую ошибку восстановления, поэтому может быть полезен как часть дальнейших моделей машинного обучения. PCA в данном случае менее эффективен из-за низкой доли объяснённой дисперсии. Основной особенностью задачи является не дисбаланс классов, а большое количество категориальных признаков и сложная нелинейная структура данных.

Вывод: научился применять автоэнкодеры для осуществления визуализации данных и их анализа.